

IBM INTERNSHIP PROJECT REPORT

AI-POWERED STUDENT ACADEMIC ASSISTANT USING AGENTIC AI

A Project Report based on the IBM Agentic AI Project Template

Submitted by: RHOGAN.M

Department: Information Technology

Institution: Pavendar Bharathidasan College of Engineering and Technology

1. Project Overview

Project Title: AI-Powered Student Academic Assistant using Agentic AI

Introduction: Artificial Intelligence is increasingly being used to support education, information retrieval, and personalized learning. This project proposes an intelligent academic assistant that uses an Agentic AI approach to understand student requests and coordinate specialized tasks.

Problem Statement: Students commonly need help with study planning, academic questions, assignment organization, learning resources, and progress tracking. These activities are often handled separately, requiring repeated searching and manual organization. The proposed system provides a single interface through which students can request multiple forms of academic support.

Brief Description: The system is designed as a multi-agent academic assistant. A coordinator agent receives the student's request, identifies the task, and routes it to an appropriate specialized agent. Depending on the request, the system can generate a study plan, answer academic questions, assist with assignments, organize information, or provide progress-related suggestions.

Need for the Project: A centralized assistant can make academic support more accessible and organized. Agentic AI is useful because the system is not limited to producing a single response; it can select an appropriate task flow and coordinate different capabilities.

2. Scope of the Project

The project focuses on an academic-support prototype for students. Its scope includes conversational interaction, task classification, specialized AI agents, study-plan generation, academic question assistance, basic progress management, and a simple user interface.

The system is intended as a learning and productivity assistant rather than a replacement for teachers, lecturers, or institutional academic policies.

3. Objectives & Proposed Solution

Project Objectives

- Develop an Agentic AI-based student assistant.
- Understand and classify common academic requests.
- Route requests to suitable specialized agents.
- Generate personalized study suggestions.

- Support question answering and assignment assistance.
- Maintain a structured workflow for student-support tasks.
- Demonstrate practical integration of AI services and application development.

Proposed Solution

The proposed solution consists of a user interface, a coordinator agent, specialized task agents, an AI/knowledge layer, and an optional data-storage layer. The user interacts with the assistant through a simple application. The coordinator determines the intent and sends the task to the relevant agent.

How the Agentic AI Solution Works

1. The student enters a request.
2. The coordinator agent interprets the request.
3. The system identifies the required task or agent.
4. The selected agent processes the request using the available AI capability, knowledge, or stored data.
5. The result is returned to the coordinator.
6. The final response is presented through the user interface.
7. Where appropriate, relevant progress information can be stored for later use.

Example Workflow

If a student asks for a seven-day preparation plan for an upcoming subject, the coordinator identifies the request as a study-planning task. The study-planner agent creates a structured plan based on the information supplied by the student. The result is then displayed in an easy-to-read format.

4. System Architecture & Key Features

System Architecture

The proposed architecture can be represented as the following logical flow:

Student → User Interface → Coordinator Agent → Specialized Agents → AI/Knowledge/Data Layer → Response → Student

The coordinator acts as the central decision component. Specialized agents can be separated according to their responsibilities, making the system easier to extend and maintain.

Major Agents

- **Coordinator Agent:** Understands the request and selects the appropriate workflow.

- **Study Planner Agent:** Generates study schedules and topic-wise preparation plans.
- **Question Assistant Agent:** Provides explanations and answers for academic queries.
- **Assignment Support Agent:** Helps organize assignment requirements and provides structured guidance.
- **Progress Agent:** Uses available progress information to provide study suggestions.

Key Features

1. Conversational academic assistance.
2. Multi-agent task coordination.
3. Study-plan generation.
4. Question-answer support.
5. Assignment organization.
6. Progress-oriented suggestions.
7. Simple and user-friendly interface.
8. Expandable architecture for future agents.

Benefits

The architecture separates responsibilities between agents, allowing individual functions to be improved without redesigning the entire application. It also provides a practical demonstration of how Agentic AI can be applied to an everyday student-support problem.

5. Implementation

Technologies / Tools Used

- Python – application and AI workflow development.
- IBM watsonx.ai – AI/LLM platform component for the proposed IBM-oriented workflow.
- IBM Cloud – cloud environment for IBM services where applicable.
- Agent framework – for coordinating agent tasks.
- Streamlit or Flask – simple application interface.
- MySQL – structured storage for student-related data where required.

Software Workflow

The application starts with the user interface. A request is passed to the coordinator. The coordinator evaluates the intent and invokes the corresponding specialized component. The specialized component processes the request and returns a structured result. The interface then displays the response.

Data Flow

User Input → Request Analysis → Agent Selection → Task Processing → Result Validation → Response Display

Development Method

The project can be developed incrementally. First, the user interface and basic request handling are created. Next, the coordinator and individual agents are added. After integration, sample queries are tested to verify that the correct agent is selected and that the output is presented clearly.

Testing Approach

Testing should include different categories of student requests, ambiguous queries, incomplete inputs, and repeated requests. The system should be checked for correct routing, understandable responses, stable application behavior, and appropriate handling of unsupported requests.

6. Results, Output & Evaluation

Expected / Demonstrated Outputs

The prototype is designed to produce outputs such as a personalized study schedule, structured answers to academic questions, assignment-support guidance, and progress-oriented recommendations.

Sample Interaction 1 – Study Planning

Input: Create a study plan for my upcoming examination.

Process: Coordinator → Study Planner Agent → Structured schedule generation.

Output: A day-wise or topic-wise study plan based on the information supplied by the student.

Sample Interaction 2 – Academic Question

Input: Explain a computer science concept in simple terms.

Process: Coordinator → Question Assistant Agent.

Output: A concise explanation with examples appropriate to the requested topic.

Sample Interaction 3 – Assignment Support

Input: Help me organize the tasks for my assignment.

Process: Coordinator → Assignment Support Agent.

Output: A structured list of tasks, suggested order, and completion checkpoints.

Results Achieved

The proposed system demonstrates how an Agentic AI architecture can centralize multiple student-support functions. The main result is a coordinated workflow in which different agents handle different tasks rather than relying on a single undifferentiated process.

Evaluation Criteria

The system can be evaluated using task-routing accuracy, response relevance, usability, response consistency, processing time, and successful completion of representative academic-support tasks.

Screenshot / Output Section

For the final internship submission, screenshots can be inserted here showing the application home screen, sample query, agent workflow, generated study plan, and result page.

7. Conclusion, Challenges & Future Scope

Project Conclusion

The AI-Powered Student Academic Assistant demonstrates the application of Agentic AI to a practical student-support scenario. By using a coordinator and specialized agents, the system can organize several academic tasks through a common interface. The project also provides a clear example of integrating AI concepts with application development.

Challenges Faced

- Designing reliable coordination between agents.
- Selecting the correct agent for different requests.
- Maintaining useful and consistent responses.
- Handling incomplete or ambiguous student inputs.
- Integrating AI services with the application interface.
- Managing structured student information responsibly.
- Testing the workflow across different types of requests.

Future Enhancements

1. Voice-based interaction.
2. Tamil and other regional-language support.
3. Calendar and timetable integration.
4. Mobile application version.

5. Improved institutional-document retrieval.
6. Personalized learning analytics.
7. Additional agents for career guidance, revision, and resource organization.
8. Stronger evaluation and monitoring of AI responses.

Overall Impact

The project can serve as a foundation for a broader intelligent learning assistant. Its modular architecture allows additional capabilities to be introduced without changing the basic interaction model.

8. References & Project Summary

References

1. IBM watsonx.ai documentation and learning resources.
2. IBM Cloud documentation and service guides.
3. Python documentation.
4. Documentation for the selected agent framework.
5. Documentation for Streamlit or Flask, depending on implementation.
6. MySQL documentation for database integration.

Project Summary

This internship project presents an AI-Powered Student Academic Assistant using Agentic AI. The system receives student requests, interprets their intent, routes them to specialized agents, processes the task, and returns an organized response. The proposed architecture includes a coordinator agent, task-specific agents, an AI/knowledge layer, an application interface, and optional data storage.

Key Takeaways

- Agentic AI can coordinate multiple specialized tasks.
- A modular architecture improves extensibility.
- Academic-support tasks can be organized through a single interface.
- IBM-oriented AI and cloud technologies can be incorporated into an internship prototype.
- The project can be extended with multilingual, voice, analytics, and mobile capabilities.

End of Project Report

Note: Before final submission, add actual implementation screenshots, your IBM internship details, mentor/faculty information, project dates, and verified references used during development.